Assignment

Kabir Kanha Arora
CSD 342 - Data Mining and Data Warehousing
Shiv Nadar University

April 22, 2020

Question 1

Download data set Iris and calculate the following:

(a) the average (mean) value for each of the four features

(b) the standard deviation for each of the features

(c) repeat steps (a) and (b) but separately for each type of flower

(d) draw four box plots, one for each feature, such that each figure shows three boxes, one for each type of flower. Properly label your axes in all box plots.

(e) Find out outliers (if any) in the data set

Data set Iris can be downloaded from *http://archive.ics.uci.edu/ml/datasets/iris*.

The reader is requested to refer to the attached Excel workbook *Assignment1-Q1.xlsx*. This workbook has multiple worksheets, that can be visited by clicking on the tab of the required worksheet. All the calculation has been done in this workbook, and I am only presenting the final answers here.

(a) The calculated mean for each of the following features is as follows:

Feature	Mean
Sepal Length	5.84
Sepal Width	3.05
Petal Length	3.76
Petal Width	1.20

(b) The calculated standard deviation for each of the following features is as follows:

Feature	Std. Deviation
Sepal Length	0.825
Sepal Width	0.432
Petal Length	1.759
Petal Width	0.761

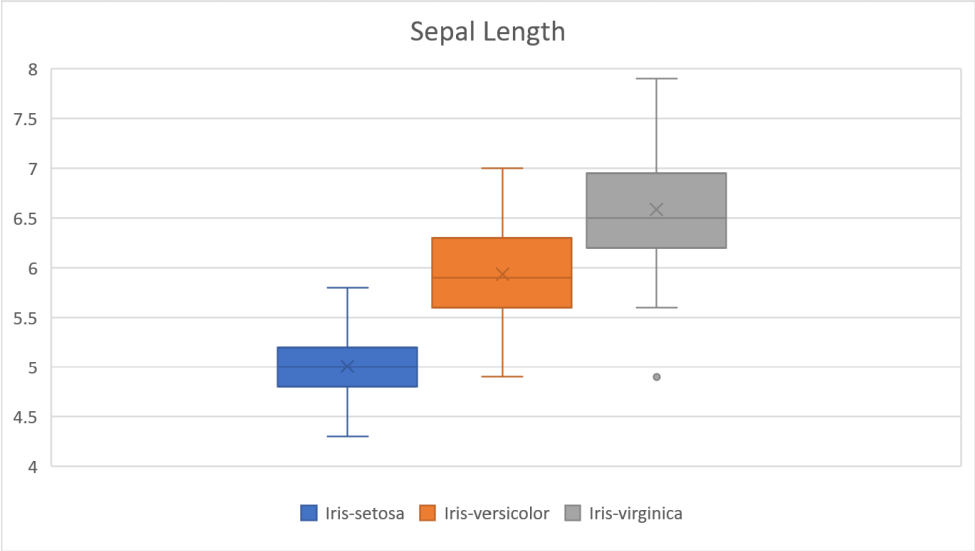
(c) The calculated mean for each of the following features, separately for each type of flower is as follows:

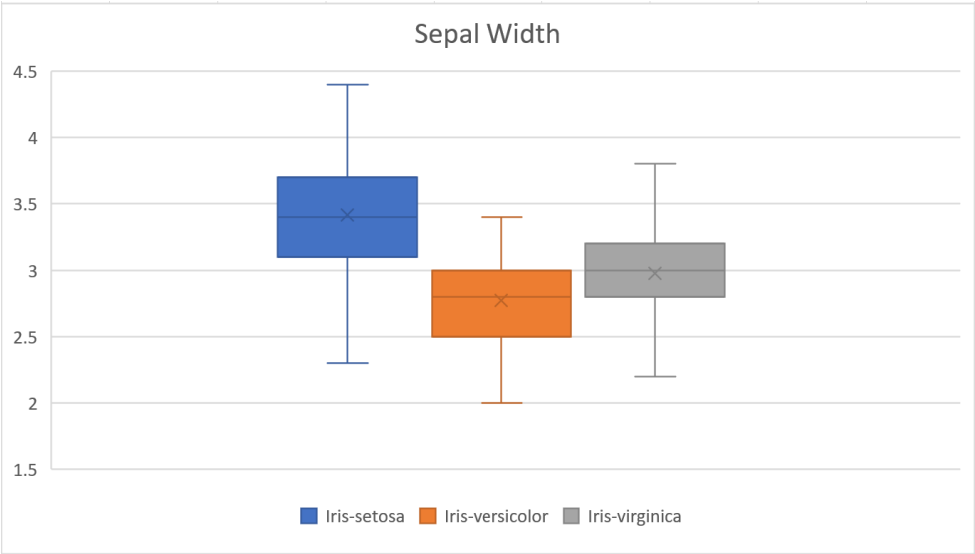
Class	Feature	Mean
Iris-setosa	Sepal Length	5.01
	Sepal Width	3.42
	Petal Length	1.46
	Petal Width	0.24
Iris-versicolor	Sepal Length	5.94
	Sepal Width	2.77
	Petal Length	4.26
	Petal Width	1.33
Iris-virginica	Sepal Length	6.59
	Sepal Width	2.97
	Petal Length	5.55
	Petal Width	2.03

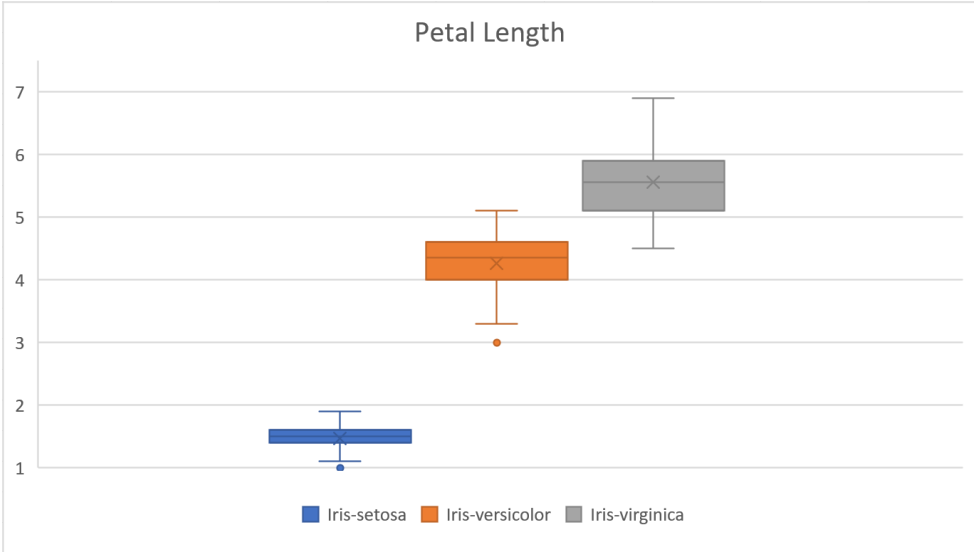
The calculated standard deviation for each of the following features, separately for each type of flower is as follows:

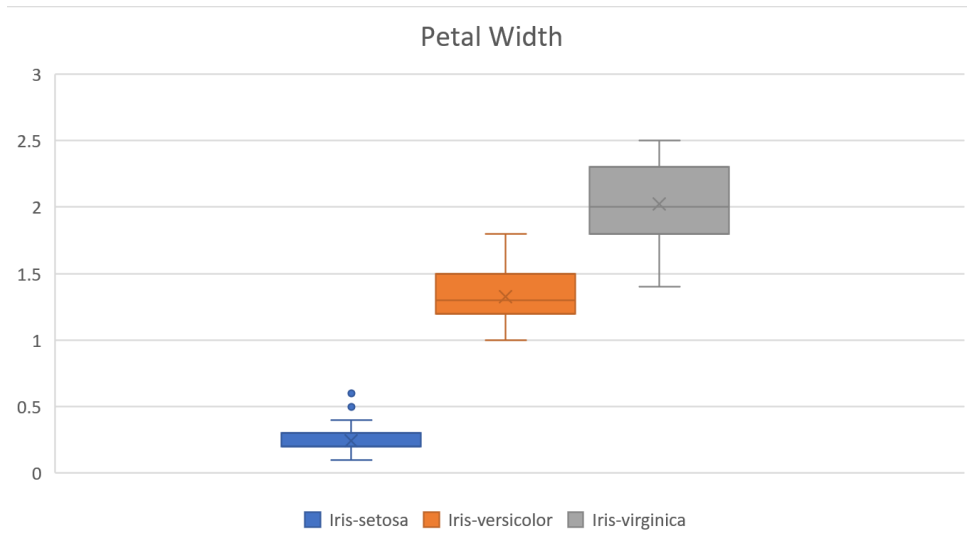
Class	Feature	Std. Deviation
Iris-setosa	Sepal Length	0.349
	Sepal Width	0.377
	Petal Length	0.172
	Petal Width	0.106
Iris-versicolor	Sepal Length	0.511
	Sepal Width	0.311
	Petal Length	0.465
	Petal Width	0.196
Iris-virginica	Sepal Length	0.629
	Sepal Width	0.319
	Petal Length	0.546
	Petal Width	0.272

(d) I have used exclusive median for calculating the quartiles for the box plots as that makes more intuitive sense to me. This technique gives fewer outliers than when inclusive median. This can easily be changed depending upon the requirements. The boxplots with outliers are shown in the figures below:


Sepal Length
8
7.5
7
6.5
6
5.5
5
4.5
4
Iris-setosa
Iris-versicolor
Iris-virginica


Sepal Width
4.5
4
3.5
3
2.5
2
1.5
Iris-setosa
Iris-versicolor
Iris-virginica


Petal Length
7
6
5
4
3
2
1
Iris-setosa
Iris-versicolor
Iris-virginica



(e) The outliers are visible are points outside the boxplots in the figures above. These have also been highlighted red in the attached Excel workbook. Outliers have been found according to the following rule: *When reviewing a box plot, an outlier is defined as a data point that is located outside the whiskers of the box plot i.e., outside* 1.5 *times the interquartile range above the upper quartile and below the lower quartile (Q1 - 1.5 * IQR or Q3 + 1.5 * IQR)*

The outliers are also summarised the the table given below:

Feature	Class	Outlier value
Sepal Length	Iris-virginica	4.9
Petal Length	Iris-setosa	1
	Iris-versicolor	3
Petal Width	Iris-setosa	0.5
	Iris-setosa	0.6

Question 2

Suppose that the data mining task is to cluster the following eight points (with (x, y) representing location) into three clusters:
$A1(2, 10), A2(2, 5), A3(8, 4), B1(5, 8), B2(7, 5), B3(6, 4), C1(1, 2), C2(4, 9)$**:**
The distance function is Euclidean distance. Suppose initially we assign $A1$, $B1$, and $C1$ as the center of each cluster, respectively. Use separately k-means, k-medoids and k-median algorithms to show the following:

1. **The three cluster centers after the first round execution**

2. **The final three clusters**

(a) **k-Means**

We are given the initial centres to be $(2, 10)$, $(5, 8$ and $(1, 2)$ and we are using the Euclidean distance as the distance function. Let us denote the set of all the data points as D and the set of the medoids(centres) as S. The initial distances of all the points to the centres look like this:

	(2,10)	(5,8)	(1,2)
(2,10)	0	$\sqrt{13}$	$\sqrt{65}$
(2,5)	$\sqrt{25}$	$\sqrt{18}$	$\sqrt{10}$
(8,4)	$\sqrt{72}$	$\sqrt{25}$	$\sqrt{53}$
(5,8)	$\sqrt{13}$	0	$\sqrt{52}$
(7,5)	$\sqrt{50}$	$\sqrt{13}$	$\sqrt{45}$
(6,4)	$\sqrt{52}$	$\sqrt{17}$	$\sqrt{28}$
(1,2)	$\sqrt{65}$	$\sqrt{52}$	0
(4,9)	$\sqrt{5}$	$\sqrt{2}$	$\sqrt{58}$

So, we see that the contents of the three clusters are now as follows:

(2,10)	(8,4)	(2,5)
	(5,8)	(1,2)
	(7,5)	
	(6,4)	
	(4,9)	

So the new cluster centres are clearly $(2, 10)$, $((8+5+7+6+4)/5, (4+8+5+4+9)/5)$ and $((2+1)/2, (5+2)/2)$ **i.e,** $(2, 10)$**,** $(6, 6)$ **and** $(1.5, 3.5)$ **after the first iteration**.

Now, the distances to the centres are as follows:

	(2,10)	(6,6)	(1.5,3.5)
(2,10)	0	$\sqrt{32}$	$\sqrt{42.5}$
(2,5)	$\sqrt{25}$	$\sqrt{17}$	$\sqrt{2.5}$
(8,4)	$\sqrt{72}$	$\sqrt{8}$	$\sqrt{42.5}$
(5,8)	$\sqrt{13}$	$\sqrt{5}$	$\sqrt{32.5}$
(7,5)	$\sqrt{50}$	$\sqrt{2}$	$\sqrt{32.5}$
(6,4)	$\sqrt{52}$	$\sqrt{4}$	$\sqrt{20.5}$
(1,2)	$\sqrt{65}$	$\sqrt{41}$	$\sqrt{2.5}$
(4,9)	$\sqrt{5}$	$\sqrt{13}$	$\sqrt{36.5}$

So, we see that the contents of the three clusters are now as follows:

(2,10)	(8,4)	(2,5)
(4,9)	(5,8)	(1,2)
	(7,5)	
	(6,4)	

So the new cluster centres are clearly $((2+4)/2, (10+9)/2)$, $((8+5+7+6)/4, (4+8+5+4)/4)$ and $((2+1)/2, (5+2)/2)$ **i.e,** $(3, 9.5)$**,** $(6.5, 5.25)$ **and** $(1.5, 3.5)$**.**

Now, the distances to the centres are as follows:

	(3,9.5)	(6.5,5.25)	(1.5,3.5)
(2,10)	$\sqrt{1.25}$	$\sqrt{42.81}$	$\sqrt{42.5}$
(2,5)	$\sqrt{21.25}$	$\sqrt{20.31}$	$\sqrt{2.5}$
(8,4)	$\sqrt{55.25}$	$\sqrt{3.81}$	$\sqrt{42.5}$
(5,8)	$\sqrt{6.25}$	$\sqrt{9.81}$	$\sqrt{32.5}$
(7,5)	$\sqrt{36.25}$	$\sqrt{0.31}$	$\sqrt{32.5}$
(6,4)	$\sqrt{39.25}$	$\sqrt{1.81}$	$\sqrt{20.5}$
(1,2)	$\sqrt{60.25}$	$\sqrt{40.81}$	$\sqrt{2.5}$
(4,9)	$\sqrt{1.25}$	$\sqrt{20.31}$	$\sqrt{36.5}$

So, we see that the contents of the three clusters are now as follows:

(2,10)	(8,4)	(2,5)
(4,9)	(7,5)	(1,2)
(5,8)	(6,4)	

So the new cluster centres are clearly $((2+4+5)/3, (10+9+8)/3)$, $((8+7+6)/3, (4+5+4)/3)$ and $((2+1)/2, (5+2)/2)$ **i.e,** $(3.67, 9)$**,** $(7, 4.33)$ **and** $(1.5, 3.5)$**.**

Now, the distances to the centres are as follows:

	(3,9.5)	(6.5,5.25)	(1.5,3.5)
(2,10)	$\sqrt{3.79}$	$\sqrt{57.15}$	$\sqrt{42.5}$
(2,5)	$\sqrt{18.79}$	$\sqrt{25.45}$	$\sqrt{2.5}$
(8,4)	$\sqrt{43.75}$	$\sqrt{1.11}$	$\sqrt{42.5}$
(5,8)	$\sqrt{2.77}$	$\sqrt{17.47}$	$\sqrt{32.5}$
(7,5)	$\sqrt{27.09}$	$\sqrt{0.45}$	$\sqrt{32.5}$
(6,4)	$\sqrt{30.43}$	$\sqrt{1.11}$	$\sqrt{20.5}$
(1,2)	$\sqrt{56.13}$	$\sqrt{41.43}$	$\sqrt{2.5}$
(4,9)	$\sqrt{0.11}$	$\sqrt{30.81}$	$\sqrt{36.5}$

So, we see that the contents of the three clusters are now as follows:

(2,10)	(8,4)	(2,5)
(4,9)	(7,5)	(1,2)
(5,8)	(6,4)	

Clearly, there has been no change in the cluster composition, so these are the **final clusters**.

(b) **k-medoids**

Now, k-medoids is a tough enough algorithm to implement as it is and without loss of the partial ordering of costs, we can use the square of the Euclidean distance as the distance function. This removes the pesky square root signs from the costs. We are given the initial centres to be $(2, 10)$, $(5, 8)$ and $(1, 2)$ and the initial costs of all the points with respect to the chosen medoids look like this:

	(2,10)	(5,8)	(1,2)
(2,10)	0	13	65
(2,5)	25	18	10
(8,4)	72	25	53
(5,8)	13	0	52
(7,5)	50	13	45
(6,4)	52	17	28
(1,2)	65	52	0
(4,9)	5	2	5

So, we see that the contents of the three clusters are now as follows:

(2,10)	(8,4)	(2,5)
	(5,8)	(1,2)
	(7,5)	
	(6,4)	
	(4,9)	

The total cost in this case is clearly $10 + 25 + 13 + 17 + 2 = \mathbf{67}$

The set $D - S$ can be represented as

D-S
(2,5)
(8,4)
(7,5)
(6,4)
(4,9)

And the set S can be represented as

S
(2,10)
(5,8)
(1,2)

Now, we need to find a swap from $D-S$ to S that will cause the best possible reduction in cost. Instead of going through all possible combinations, let us try a random swap. Let us swap $(7, 5)$ with $(5, 8)$. **Hence, the new cluster centres after the first iteration are** $(2, 10)$**,** $(7, 5)$ **and** $(1, 2)$

The cost table now looks like this:

	(2,10)	(7,5)	(1,2)
(2,10)	0	50	65
(2,5)	25	25	10
(8,4)	72	2	53
(5,8)	13	13	52
(7,5)	50	0	45
(6,4)	52	2	28
(1,2)	65	45	0
(4,9)	5	25	58

The total cost in this case is clearly $10 + 2 + 13 + 2 + 5 = \mathbf{32}$, which is a massive reduction.

The set $D - S$ can now be represented as

D-S
(2,5)
(8,4)
(5,8)
(6,4)
(4,9)

And the set S can be represented as

S
(2,10)
(7,5)
(1,2)

Let us try another random swap. Let us try swapping $(2, 10)$ with $(2, 5)$. The cost table would now look like this:

	(2,5)	(7,5)	(1,2)
(2,10)	25	50	65
(2,5)	0	25	10
(8,4)	37	2	53
(5,8)	18	13	52
(7,5)	25	0	45
(6,4)	17	2	28
(1,2)	10	45	0
(4,9)	20	25	58

The total cost in this case is clearly $25 + 2 + 13 + 2 + 20 = \mathbf{62}$, which is worse than 32. So we do not go for this swap.

But we cannot say that the solution is reached, since there are other swaps possible.

Let's try another swap. Let's swap $(8, 4)$ with $(2, 10)$. The cost table would now look like this:

	(8,4)	(7,5)	(1,2)
(2,10)	72	50	65
(2,5)	37	25	10
(8,4)	0	2	53
(5,8)	25	13	52
(7,5)	2	0	45
(6,4)	4	2	28
(1,2)	53	45	0
(4,9)	41	25	58

The total cost in this case is clearly $50 + 10 + 13 + 2 + 25 = \mathbf{100}$, which is worse than 32. So we do not go for this swap.

Let's try another swap. Let's swap $(5, 8)$ with $(2, 10)$. The cost table would now look like this:

	(5,8)	(7,5)	(1,2)
(2,10)	13	50	65
(2,5)	18	25	10
(8,4)	25	2	53
(5,8)	0	13	52
(7,5)	13	0	45
(6,4)	17	2	28
(1,2)	52	45	0
(4,9)	2	25	58

The total cost in this case is clearly $13 + 10 + 2 + 2 + 2 = \mathbf{29}$, which is better than 32. So we do go for this swap.

The set $D - S$ can now be represented as

D-S
(2,5)
(8,4)
(2,10)
(6,4)
(4,9)

And the set S can be represented as

S
(5,8)
(7,5)
(1,2)

Note that we still need to check if a swap can further reduce. We can continue the process, but we see that swapping $(5, 8)$ with any of the first three entries from $D - S$

will give a table that we have computed previously, so we start from the fourth entry of $D - S$.

Let's swap $(5, 8)$ with $(6, 4)$. The cost table would now look like this:

	(6,4)	(7,5)	(1,2)
(2,10)	52	50	65
(2,5)	17	25	10
(8,4)	4	2	53
(5,8)	17	13	52
(7,5)	2	0	45
(6,4)	0	2	28
(1,2)	29	45	0
(4,9)	29	25	58

The total cost in this case is clearly $50 + 10 + 2 + 13 + 25 = \mathbf{100}$, which is worse than 29. So we do not go for this swap.

Let's swap $(5, 8)$ with $(4, 9)$. The cost table would now look like this:

	(4,9)	(7,5)	(1,2)
(2,10)	5	50	65
(2,5)	20	25	10
(8,4)	41	2	53
(5,8)	2	13	52
(7,5)	25	0	45
(6,4)	29	2	28
(1,2)	58	45	0
(4,9)	0	25	58

The total cost in this case is clearly $5 + 10 + 2 + 2 + 2 = \mathbf{21}$, which is better than 29. So we do go for this swap.

The set $D - S$ can now be represented as

D-S
(2,5)
(8,4)
(2,10)
(6,4)
(5,8)

And the set S can be represented as

S
(4,9)
(7,5)
(1,2)

We must continue until all possible swaps are exhausted.

Let's swap $(7, 5)$ with $(2, 5)$. The cost table would now look like this:

	(4,9)	(2,5)	(1,2)
(2,10)	5	25	65
(2,5)	20	0	10
(8,4)	41	37	53
(5,8)	2	18	52
(7,5)	25	25	45
(6,4)	29	17	28
(1,2)	58	10	0
(4,9)	0	20	58

The total cost in this case is clearly $5 + 37 + 18 + 25 + 17 = \mathbf{86}$, which is worse than 21. So we do not go for this swap.

Let's swap $(7, 5)$ with $(8, 4)$. The cost table would now look like this:

	(4,9)	(8,4)	(1,2)
(2,10)	5	62	65
(2,5)	20	37	10
(8,4)	41	0	53
(5,8)	2	25	52
(7,5)	25	2	45
(6,4)	29	4	28
(1,2)	58	53	0
(4,9)	0	41	58

The total cost in this case is clearly $5 + 10 + 2 + 2 + 4 = \mathbf{23}$, which is worse than 21. So we do not go for this swap.

Let's swap $(7, 5)$ with $(2, 10)$. The cost table would now look like this:

	(4,9)	(2,10)	(1,2)
(2,10)	5	52	65
(2,5)	20	17	10
(8,4)	41	4	53
(5,8)	2	17	52
(7,5)	25	2	45
(6,4)	29	0	28
(1,2)	58	29	0
(4,9)	0	29	58

The total cost in this case is clearly $5 + 10 + 14 + 2 + 2 = \mathbf{23}$, which is worse than 21. So we do not go for this swap.

Let's swap $(7, 5)$ with $(6, 4)$. The cost table would now look like this:

	(4,9)	(6,4)	(1,2)
(2,10)	5	0	65
(2,5)	20	25	10
(8,4)	41	72	53
(5,8)	2	13	52
(7,5)	25	50	45
(6,4)	29	52	28
(1,2)	58	65	0
(4,9)	0	5	58

The total cost in this case is clearly $10 + 41 + 2 + 2 + 25 + 29 = \mathbf{107}$, which is worse than 21. So we do not go for this swap.

Let's swap $(7, 5)$ with $(5, 8)$. The cost table would now look like this:

	(4,9)	(5,8)	(1,2)
(2,10)	5	13	65
(2,5)	20	18	10
(8,4)	41	25	53
(5,8)	2	0	52
(7,5)	25	13	45
(6,4)	29	17	28
(1,2)	58	52	0
(4,9)	0	2	58

The total cost in this case is clearly $5 + 10 + 25 + 13 + 17 = \mathbf{70}$, which is worse than 21. So we do not go for this swap.

Moving on to the last medoid.

Let's swap $(1, 2)$ with $(2, 5)$. The cost table would now look like this:

	(4,9)	(7,5)	(2,5)
(2,10)	5	50	25
(2,5)	20	25	0
(8,4)	41	2	37
(5,8)	2	13	18
(7,5)	25	0	25
(6,4)	29	2	17
(1,2)	58	45	10
(4,9)	0	25	20

The total cost in this case is clearly $5 + 2 + 2 + 2 + 10 = \mathbf{21}$, which is as good as 21. So we may go for this swap. Let us note it down and keep it, but not make the swap.

Let's swap $(1, 2)$ with $(8, 4)$. The cost table would now look like this:

	(4,9)	(7,5)	(8,4)
(2,10)	5	50	72
(2,5)	20	25	37
(8,4)	41	2	0
(5,8)	2	13	25
(7,5)	25	0	2
(6,4)	29	2	4
(1,2)	58	45	53
(4,9)	0	25	41

The total cost in this case is clearly $5 + 20 + 2 + 2 + 45 = \mathbf{74}$, which is worse than 21. So we do not go for this swap.

Let's swap $(1, 2)$ with $(2, 10)$. The cost table would now look like this:

	(4,9)	(7,5)	(2,10)
(2,10)	5	50	0
(2,5)	20	25	25
(8,4)	41	2	72
(5,8)	2	13	13
(7,5)	25	0	50
(6,4)	29	2	52
(1,2)	58	45	65
(4,9)	0	25	5

The total cost in this case is clearly $20 + 2 + 2 + 2 + 45 = \mathbf{71}$, which is worse than 21. So we do not go for this swap.

Let's swap $(1, 2)$ with $(6, 4)$. The cost table would now look like this:

	(4,9)	(7,5)	(6,4)
(2,10)	5	50	52
(2,5)	20	25	17
(8,4)	41	2	4
(5,8)	2	13	17
(7,5)	25	0	2
(6,4)	29	2	0
(1,2)	58	45	29
(4,9)	0	25	29

The total cost in this case is clearly $5 + 17 + 2 + 2 + 29 = \mathbf{55}$, which is worse than 21. So we do not go for this swap.

Let's swap $(1, 2)$ with $(5, 8)$. The cost table would now look like this:

	(4,9)	(7,5)	(5,8)
(2,10)	5	50	13
(2,5)	20	25	18
(8,4)	41	2	25
(5,8)	2	13	0
(7,5)	25	0	13
(6,4)	29	2	17
(1,2)	58	45	52
(4,9)	0	25	2

The total cost in this case is clearly $5 + 18 + 2 + 2 + 45 = \mathbf{72}$, which is worse than 21. So we do not go for this swap.

At this point, I'd like to give myself a pat on the back for grinding through this excruciating exercise. We observe that there were two possible configurations that gave us the minimum cost.

They are:

S
(4,9)
(7,5)
(1,2)

and

S
(4,9)
(7,5)
(2,5)

They both have a minimum cost of $\sqrt{5} + \sqrt{2} + \sqrt{2} + \sqrt{2} + \sqrt{10} \approx 9.64$. Note that we have re-introduced the square roots that we had taken off for convenience earlier. The answer cannot be incorrect as we only used the costs for comparison, and squaring is an ordinal transformation, i.e., it preserves the partial ordering of elements.

The clusters corresponding to both two sets of medoids are:

(2,10)	(8,4)	(2,5)
(5,8)	(7,5)	(1,2)
(4,9)	(6,4)	

and this is the optimal answer for the given set of points.

(c) **k-medians**

We are given the initial centres to be $(2, 10)$, $(5, 8)$ and $(1, 2)$ and the initial costs of all the points with respect to the chosen centres look like this:

	(2,10)	(5,8)	(1,2)
(2,10)	0	$\sqrt{13}$	$\sqrt{65}$
(2,5)	$\sqrt{25}$	$\sqrt{18}$	$\sqrt{10}$
(8,4)	$\sqrt{72}$	$\sqrt{25}$	$\sqrt{53}$
(5,8)	$\sqrt{13}$	0	$\sqrt{52}$
(7,5)	$\sqrt{50}$	$\sqrt{13}$	$\sqrt{45}$
(6,4)	$\sqrt{52}$	$\sqrt{17}$	$\sqrt{28}$
(1,2)	$\sqrt{65}$	$\sqrt{52}$	0
(4,9)	$\sqrt{5}$	$\sqrt{2}$	$\sqrt{58}$

So, we see that the contents of the three clusters are as follows:

(2,10)	(8,4)	(2,5)
	(5,8)	(1,2)
	(7,5)	
	(6,4)	
	(4,9)	

Clearly, the new centres after the first iteration are $(2, 10)$**,** $(6, 5)$ **and** $(1.5, 3.5)$

The cost table now looks like this:

	(2,10)	(6,5)	(1.5,3.5)
(2,10)	0	$\sqrt{41}$	$\sqrt{42.5}$
(2,5)	$\sqrt{25}$	$\sqrt{16}$	$\sqrt{2.5}$
(8,4)	$\sqrt{72}$	$\sqrt{5}$	$\sqrt{42.5}$
(5,8)	$\sqrt{13}$	$\sqrt{10}$	$\sqrt{32.5}$
(7,5)	$\sqrt{50}$	$\sqrt{1}$	$\sqrt{32.5}$
(6,4)	$\sqrt{52}$	$\sqrt{1}$	$\sqrt{20.5}$
(1,2)	$\sqrt{65}$	$\sqrt{34}$	$\sqrt{2.5}$
(4,9)	$\sqrt{5}$	$\sqrt{20}$	$\sqrt{36.5}$

So, we see that the contents of the three clusters are now as follows:

(2,10)	(8,4)	(2,5)
(4,9)	(5,8)	(1,2)
	(7,5)	
	(6,4)	

Clearly, the new medians are $(3, 9.5)$, $(6.5, 4.5)$ and $(1.5, 3.5)$

The cost table now looks like this:

	(3,9.5)	(6.5,4.5)	(1.5,3.5)
(2,10)	$\sqrt{1.25}$	$\sqrt{50.5}$	$\sqrt{42.5}$
(2,5)	$\sqrt{21.25}$	$\sqrt{20.5}$	$\sqrt{2.5}$
(8,4)	$\sqrt{55.25}$	$\sqrt{2.5}$	$\sqrt{42.5}$
(5,8)	$\sqrt{6.25}$	$\sqrt{14.5}$	$\sqrt{32.5}$
(7,5)	$\sqrt{36.25}$	$\sqrt{0.5}$	$\sqrt{32.5}$
(6,4)	$\sqrt{39.25}$	$\sqrt{0.5}$	$\sqrt{20.5}$
(1,2)	$\sqrt{60.25}$	$\sqrt{36.5}$	$\sqrt{2.5}$
(4,9)	$\sqrt{1.25}$	$\sqrt{26.5}$	$\sqrt{36.5}$

So, we see that the contents of the three clusters are now as follows:

(2,10)	(8,4)	(2,5)
(4,9)	(6,4)	(1,2)
(5,8)	(7,5)	

Clearly, the new medians are $(4, 9)$, $(7, 4)$ and $(1.5, 3.5)$

The cost table now looks like this:

	(4,9)	(7,4)	(1.5,3.5)
(2,10)	$\sqrt{5}$	$\sqrt{61}$	$\sqrt{42.5}$
(2,5)	$\sqrt{20}$	$\sqrt{26}$	$\sqrt{2.5}$
(8,4)	$\sqrt{41}$	$\sqrt{1}$	$\sqrt{42.5}$
(5,8)	$\sqrt{2}$	$\sqrt{20}$	$\sqrt{32.5}$
(7,5)	$\sqrt{25}$	$\sqrt{1}$	$\sqrt{32.5}$
(6,4)	$\sqrt{29}$	$\sqrt{1}$	$\sqrt{20.5}$
(1,2)	$\sqrt{58}$	$\sqrt{40}$	$\sqrt{2.5}$
(4,9)	$\sqrt{0}$	$\sqrt{34}$	$\sqrt{36.5}$

At this point, we notice that the clusters do not change, and so the algorithm is complete.

This clusters finally look like this:

(2,10)	(8,4)	(2,5)
(4,9)	(6,4)	(1,2)
(5,8)	(7,5)	

Question 3

For the $k - means$ algorithm, it is interesting to note that by choosing the initial cluster centers carefully, we may be able to not only speed up the algorithm's convergence, but also guarantee the quality of the final clustering. The $k - means + +$ algorithm is a variant of $k - means$, which chooses the initial centers as follows:

First, it selects one center uniformly at random from the objects in the data set. Iteratively, for each object p other than the chosen center, it chooses an object as the new center. This object is chosen at random with probability proportional to $(dist(p))^2$, where $dist(p)$ is the distance from p to the closest center that has already been chosen. The iteration continues until k centers are selected. Explain why this method will not only speed up the convergence of the $k-means$ algorithm, but also guarantee the quality of the final clustering results.

The $k-means$ algorithm has at least two major theoretic shortcomings:

- The worst case running time of the algorithm is super-polynomial in the input size.
- The approximation found can be arbitrarily bad with respect to the objective function compared to the optimal clustering.

The $k-means++$ algorithm takes care of the second problem by specifying an algorithm to initialize the cluster centers before doing the standard $k-means$ optimization iterations. With the $k-means++$ initialization, the algorithm is guaranteed to find a solution that is $\mathcal{O}(logk)$ competitive to the optimal $k-means$ solution.

To show the $k-means$ algorithm can perform arbitrarily poorly with respect to the objective function of minimizing SSE of cluster points to their respective centroids, take the example of four points in $\mathbb{R}^2$ that form an rectangle whose breadth is larger than its height.

If $k = 2$ and the two centroids initially lie at the midpoints of the top and bottom line segments of the rectangle, the algorithm converges immediately, without any change. So, the two bottom data points are clustered together and the top two data points are clustered together, thereby resulting in a sub-optimal clustering because the width of the rectangle is greater than its height.

Now, we stretch the rectangle horizontally to an arbitrary width. The k-means algorithm continues to cluster the points sub-optimally, and we can make the algorithm perform arbitrarily poorly with respect to the $k-means$ objective function, which is really bad news.

The reasoning behind the $k-means++$ algorithm is that spreading out the k initial centroids is a great thing. The seeding method specified in the question yields a lot of improvement in the final error of $k-means$. Although the prepossessing time is high, the $k-means$ part itself runs very quickly and hence the algorithm actually lowers the running time.
Also, the $k-means++$ algorithm guarantees an approximation ratio O(log k) over the randomness of the algorithm. This is in opposition to $k-means$, which can generate clusterings arbitrarily worse than the optimum, as discussed earlier

Conclusion:
The reason why we can guarantee the quality of the final clustering results is because we are also choosing centroids from within the data points, at the end of the day. These centroids could well have been picked by the $k-means$ algorithm too. The only difference between these two algorithms is in the way they pick centres. $k-means$ does it absolutely randomly

and the centroids we pick in $k - means++$ are perfectly valid for $k - means$ too. So at worst, we will perform as badly as $k - means$ but never worse.
It is clear that choosing clusters centres far away is better to start off with, as otherwise we could have all the initial centroids close by and a lot of adjustments would be required to get them to the optimal places for convergence.

Question 4

Given two objects represented by the tuples $(22, 1, 42, 10)$ **and** $(20, 0, 36, 8)$**:**

(a) Compute the *Euclidean distance* **between the two objects.**

(b) Compute the *Manhattan distance* **between the two objects.**

(c) Compute the *Minkowski distance* **between the two objects, using** $q = 3$**.**

(a) The **Euclidean distance** between two objects x and y denoted by $(x_1, x_2, ..., x_n)$ and $(y_1, y_2, ..., y_n)$ is given by $\sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + ... + (x_n - y_n)^2}$

According to the question, $x = (22, 1, 42, 10)$ and $y = (20, 0, 36, 8)$.

So clearly the Euclidean distance can be written as:

$$\sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2 + (x_4 - y_4)^2}$$
$$= \sqrt{(22 - 20)^2 + (1 - 0)^2 + (42 - 36)^2 + (10 - 8)^2}$$
$$= \sqrt{(2)^2 + (1)^2 + (6)^2 + (2)^2}$$
$$= \sqrt{4 + 1 + 36 + 4}$$
$$= \sqrt{45}$$
$$= 3\sqrt{5}$$
$$\approx \mathbf{6.71}$$

(b) The **Manhattan distance** between two objects x and y denoted by $(x_1, x_2, ..., x_n)$ and $(y_1, y_2, ..., y_n)$ is given by $\mid x_1 - y_1 \mid + \mid x_2 - y_2 \mid + ... + \mid x_n - y_n \mid$

According to the question, $x = (22, 1, 42, 10)$ and $y = (20, 0, 36, 8)$.

So clearly the Manhattan distance can be written as:

$$\mid x_1 - y_1 \mid + \mid x_2 - y_2 \mid + \mid x_3 - y_3 \mid + \mid x_4 - y_4 \mid$$
$$= \mid 22 - 20 \mid + \mid 1 - 0 \mid + \mid 42 - 36 \mid + \mid 10 - 8 \mid$$
$$= \mid 2 \mid + \mid 1 \mid + \mid 6 \mid + \mid 2 \mid$$
$$= 2 + 1 + 6 + 2$$
$$= \mathbf{11}$$

(c) The **Minkowski distance** between two objects x and y denoted by $(x_1, x_2, ..., x_n)$ and $(y_1, y_2, ..., y_n)$ is given by $\sqrt[q]{\mid x_1 - y_1 \mid^q + \mid x_2 - y_2 \mid^q + ... + \mid x_n - y_n \mid^q}$ for a given parameter q. It is interesting to note that the Manhattan and Euclidean distances are special cases of Minkowski distance, with $q = 1$ and 2 respectively.

According to the question, $x = (22, 1, 42, 10)$ and $y = (20, 0, 36, 8)$.

So clearly the Minkowski distance, using $q = 3$, can be written as:

$$\sqrt[3]{\mid x_1 - y_1 \mid^3 + \mid x_2 - y_2 \mid^3 + \mid x_3 - y_3 \mid^3 + \mid x_4 - y_4 \mid^3}$$

$$= \sqrt[3]{\mid 22 - 20 \mid^3 + \mid 1 - 0 \mid^3 + \mid 42 - 36 \mid^3 + \mid 10 - 8 \mid^3}$$

$$= \sqrt[3]{\mid 2 \mid^3 + \mid 1 \mid^3 + \mid 6 \mid^3 + \mid 2 \mid^3}$$

$$= \sqrt[3]{2^3 + 1^3 + 6^3 + 2^3}$$

$$= \sqrt[3]{8 + 1 + 216 + 8}$$

$$= \sqrt[3]{233}$$

$$\approx \mathbf{6.15}$$

Question 5

Explain the $DBSCAN$ algorithm.
Show 3 examples where $DBSCAN$ is a better choice compared to $k - means$.

$DBSCAN$ is an acronym *Density-based spatial clustering of applications with noise.* It is a non-parametric algorithm, which uses density-based clustering. Given a data set of points, it clusters together points that are densely packed, and marks as *outliers* points in low density areas.
Density characteristics of data points are used to cluster them. Individual dense region data points are used to create the clusters after their density is used to classify them. This algorithm should be used to find patterns in data that are tough to find visually but that can be helpful to predict trends.
Let us define some notation:

$$Data : D$$

$$Radius : \varepsilon$$

$$Density : Minpts$$

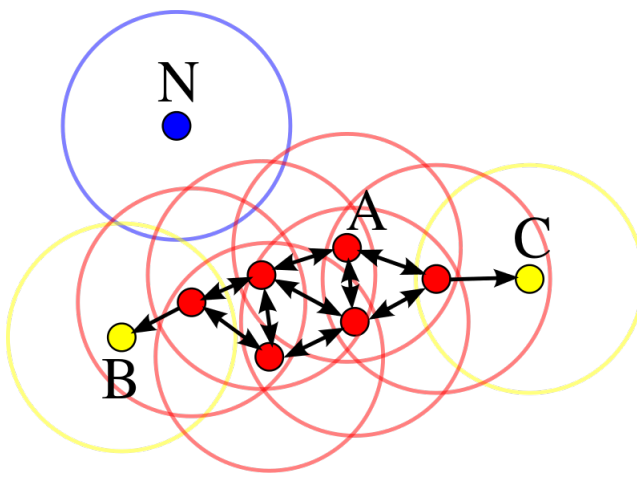
The algorithm works like this:

- Determine core, border and noise points of D at level (ε, Minpts)
- Create a graph in which core points are connected if they are within ε of one another
- Determine the connected components in the graph

- Assign each border point to connected component with which it is best connected
- Return points in each connected component as a cluster

The *density* of a data point is defined by the number of points that lie within a radius ε of that point (including the point itself). The densities of these spherical regions are used to classify the data points into *core*, *border*, or *noise* points. These notions are defined as follows:

- A data point is defined as a **core** point, if it contains at least $Minpts$ data points with a radius ε.
- A data point is defined as a **border** point, if it contains less than $Minpts$ points, but it also contains at least one core point within a radius ε.
- A data point that is neither a core point nor a border point is defined as a **noise** point



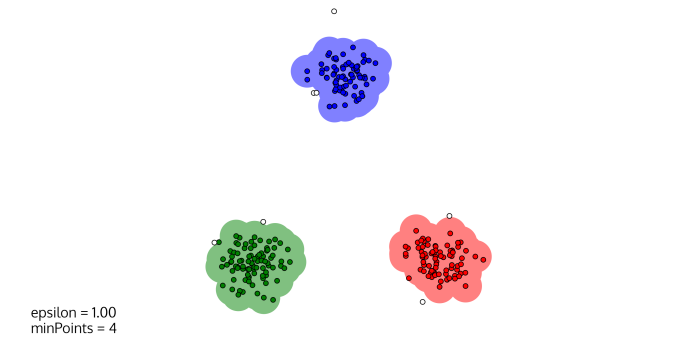
In this diagram, $Minpts = 4$. Point A and the other red points are core points, because the area surrounding these points in an ε radius contain at least 4 points (including the point itself). Because they are all reachable from one another, they form a single cluster. Points B and C are not core points, but are reachable from A (via other core points) and thus belong to the cluster as well, and are the border points. Point N is a noise point that is neither a core point nor a border point.

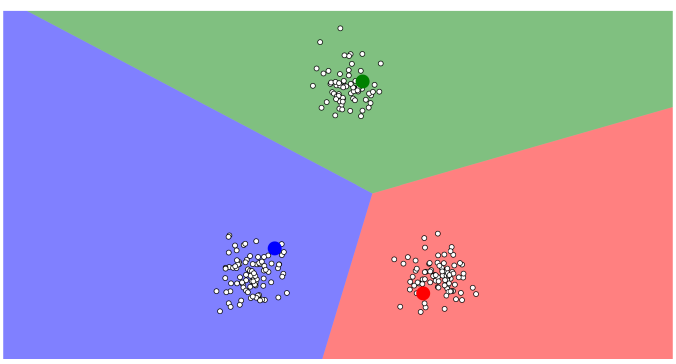
The following are examples where $DBSCAN$ is a better choice compared to $k - means$:

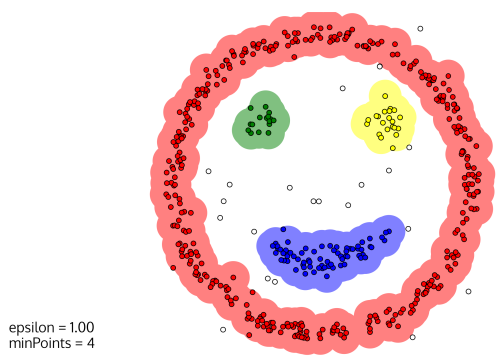
1. $DBSCAN$ does not require one to specify the number of clusters in the data beforehand, as opposed to $k - means$. So, $DBSCAN$ is a better choice when we don't know the number of clusters in the beginning.
2. $DBSCAN$ has a notion of noise, and is not so affected by outliers, while $k - means$ is extremely sensitive to outliers, as it uses Euclidean distance. So, $DBSCAN$ is a better choice when the data is likely to have outliers.

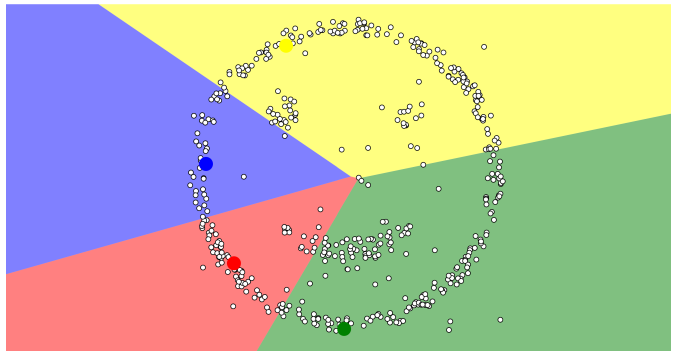
3. Curse of dimensionality affects $k-means$ since it uses distance measure, while $DBSCAN$ used density as a metric. So, $DBSCAN$ is a better choice when the number of data points is huge.

4. $K - means$ doesnt work well for non-globular structures, while $DBSCAN$ can find clusters of any shape.









Question 6

Refer to the paper attached along with the assignment and answer the following questions.

(a) **What are the challenges in performing clustering over large dynamic networks?**

(b) **Explain a real life application of the problem discussed in the paper.**

(c) **Explain the static P2P k-means algorithm and show how the algorithm can be modified in a dynamic environment.**

(d) **How are the results analyzed and what can you conclude from the paper.**

(e) **Based on the future work suggest modifications in the algorithm that may improve performance.**

(a) We know that there are two types of data - **dynamic** and **static**. Let us look at the challenges while dealing with the latter first, since they are more prominent and obvious. By its very definition, dynamic data is that data that keeps changing. The topology of the network can never be said to be fixed or finalised. The algorithm has to keep re-clustering all the time as data enters or leaves. Peers cannot even be synchronised for searching. Even for static data, this is a very hard problem to tackle. Well thought-out and clear heuristics need to be applied to solve the problem efficiently, and optimal values for the parameters required for clustering may not be found easily. This network could span continents in real life and the energy required for communication support is just phenomenal.

(b) Let us consider a real life example. Let us take **Limewire**, which was a popular P2P music sharing network. Limewire could easily be modified to adapt to this problem. Here, there is no client or server, and everyone is a *peer*. This is a sub-problem of file sharing, but gained immense popularity since everyone wants music, and it allowed

people to get music for free. A global clustering of the songs allows individual peers to produce a compact summary of their files held and they can then communicate this to their peers. These summaries are then used by other peers to more efficiently route search queries. To construct the summary structure of each peer, a distributed $k-means$ clustering is performed over all images in the network. At the end, each peer receives the correct global centroids. This can be used to find songs that are similar to your taste.

(c) **Static P2P $k-means$ algorithm**

The algorithm starts at a single node, and it generates a set of centroids and sends it to all its neighbours to start iteration one. When a node gets the centroids and γ for the first time, it sends them to all its other neighbours and starts iteration one. At some point, all the nodes enter their first iteration with the same initial centroid and γ, which is the termination threshold.

To summarise, this algorithm does repeated iterations of our modified $k-means$ algorithm for each node. For every iteration, each node polls all its neighbours for their cluster counts and centroids, which are used in the next iteration. If these are very different from the old centroids, the node goes into a *terminated* state.

Modification to dynamic environment

- *Node failure or topology change*

  If a node exits the network, its neighbours will come to know. Similarly, if an edge leaves the network, the two nodes it connects will come to know.Each node that loses a neighbour removes it from *Wait* and removes any poll messages from it. If a new edge enters the network, all the nodes it connects will find out about this, and find out their new neighbours. What we do is that we have all associated nodes wait till the next iteration before considering any new neighbours.

- *Node addition*

  When a new node enters the network, things are slightly more complicated. The node has to synchronise itself with all its neighbours. All its neighbours which are not terminated, would wait for their next iteration before polling it. But immediate neighbours that were terminated should consider becoming active again. They poll the new node for its local centroids and cluster counts, and the node updates its centroids. If they have changed significantly, the node becomes active again and informs its neighbours.

- *Data change*

  If the data in a non terminated node changes, nothing needs to be done. But is the node is terminated, we need to decide whether it should become active again or not. The node calculates its local centroids and cluster counts and checks the same for its neighbours. After updating, if the change is significant, the node becomes active again. Neighbours are, however, always informed in case there is a data change.

(d) For the experiment, the authors vary the number of nodes upto 1000 and set γ, the termination parameter to 0.01. Each node has the same centroids to start with, this removes delay in propagation.

Static Environment

Here, there were no changes in the network or data. For v nodes, they generate a data set with $649v$ points. This is called X. A centralised $k - means$ algorithm is run on X. We consider two sub-cases within this situation - namely uniform and non-uniform distribution. When the distribution is uniform, the number of data points at each node is the same. We follow a Zipfian distribution when the distribution is non-uniform. The performance of their algorithm is compared to that of $k - means$.

They define accuracy as the difference between the arrangements of clusters of centralised $k - means$ and that of the algorithm they run. For every point, they check whether the assignment matches tha of the algorithm. *Percentage Member Mismatch*, or PMM is calculated for every node. This is done for all the points in a node. It is defined as the number of points that have different assignments dividede by the total number of points that the node has. They also calculate the average and standard deviation once every node has terminated. The error is quite low.

Dynamic Environment For a dynamic environment, to make the problems somewhat tractable, they assume that the topology/data changes only every τ ticks of the global clock. Here, the two types of distribution are - stationary and non-stationary. In stationary distribution, the replaced data is from the same distribution. In non-stationary distribution, this is not what happens. The testing procedure is the same as the static environment, and initially they set 10% of the nodes as inactive. When τ ticks are done, inactive nodes are activated and they calculate accuracy. Change in the data is done according to the distribution, and they find the PMM, average and standard deviation like they did for the static case. Scalability is also checked when the size of the period is fixed. According to the results, the size of the network has no bearing on the performance.

Conclusions There is a whole lot of data, and so a centralised algorithm doesn't work too well. The $P2P$ algorithm takes into account local synchronisation of the data at a node. In both environments, they get high accuracy and small deviation. The nature of the networks in the world today is highly dynamic. This nature is built into their static algorithm so that it is not affected by the changes in topology. The bottom line is the $P2P$ does pretty well and has good accuracy as well as being a great replacement for the $k - means$ algorithm.

(e) **Future Work**

- If they assign the data points in a better fashion, communication cost could have been brought down. This is a major sore point. This can also be done by doing the local synchronisation more efficiently.

- The number of bytes that a node receives increases with the network size, but slowly.
- They can use distribution hashtables where hierarchies may be looked up with a *lookup service*. This should further reduce costs.
- The data assignment function can be improved.
- An efficient function for distribution may be used.
- This algorithm hould be tried out on a real-life problem to verify its efficacy.

References

- https://en.wikipedia.org/wiki/DBSCAN
- https://en.wikipedia.org/wiki/Minkowski_distance
- https://towardsdatascience.com/understanding-boxplots-5e2df7bcbd51
- https://en.wikipedia.org/wiki/K-medoids
- https://en.wikipedia.org/wiki/K-means%2B%2B